Random Walks with Erasure

Diversifying Personalized Ranking

Paper: Bibek Paudel (Stanford) · Abraham Bernstein (UZH) — WWW 2021

A slide-by-slide walkthrough mapped to this implementation.

Deck recreated from the talk; diagrams + code-mapping by the project.

The problem: filter bubbles

- Recommenders show "more of what you already like" → filter bubbles.
- 2010–13: social media praised as an enabler of democracy and protest.
- Then criticised for increasing polarization and harming society.
- High polarization → suspicion, hostility, and lower political participation.

→ In the code: motivation captured in GUIDE.md §1

How to bridge the divide?

- Cross-cutting awareness = exposure to viewpoints from the other side.
- It is linked to greater political understanding and participation.
- Goal: recommend content that is relevant AND bridges viewpoints.

→ In the code: RWEB (bridging) + extensions satisfaction.py / agent_sim.py

Challenge of political content

- Naive fix: mix outlets by their known slant (e.g. AllSides). Not enough.
- Two articles from the SAME outlet were shared by opposite Brexit camps.
- Ideological positions are issue/event-specific → must be learned, not labelled.

→ In the code: IdeologyModel.fit(R, S) learns issue-specific positions from behaviour

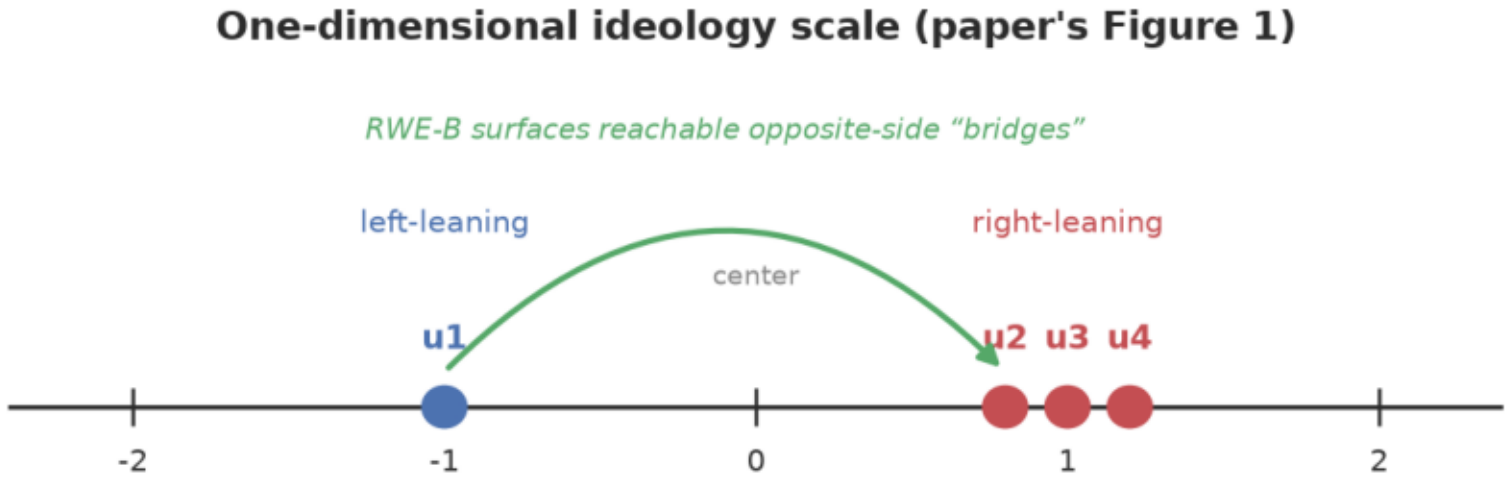
Our approach — three steps

- 1. Identify ideological positions of elites AND content from event discussions.
- 2. Choose a diversification strategy (naive "mix both sides" may fail).
- 3. Random Walk with Erasure (RWE): recommend diverse content via weak ties.

→ In the code: ideology.py · RWED / RWEB · RWE

Ideology of users, elites & content

- Each retweet yields three roles: User (retweeter), Elite (author), Content (URL).
- Two graphs: elite-endorsement R and content-share S.
- Place users, elites and content on one shared left–right scale.



→ In the code: IdeologyModel.fit(R, S) → positions θ (users), φ (elites), ψ (content)

Ideal-point model + joint learning

- Elite: $p(R=1) = \sigma(-\|\theta_u - \varphi_e\|^2 + \alpha_u + \beta_e)$
- Content: $p(S=1) = \sigma(-\|\theta_u - \psi_c\|^2 + \alpha_u + \gamma_c)$
  - Closer ideology ⇒ more likely to endorse.
- Joint objective: maximise $\mu \cdot \Sigma_R[\cdot] + \Sigma_S[\cdot] - (\lambda/2)(\|\theta\|^2 + \|\varphi\|^2 + \|\psi\|^2)$
- $\Longrightarrow$ sim(u, c): similarity of political stance.

→ In the code: ideology.py: Pi_R / Pi_S (eqs 6/9), _objective (eq 11); sim = RWEB.similarity

Estimated positions (results)

- Recovers left/right correctly across UK, US and Germany.
- Captures issue-specific cases (e.g. Conservatives who backed Remain → left).
- Joint learning (elite + content) beats elite-endorsement alone.

→ In the code: validated on synthetic data (real Twitter data not shareable) — demo_synthetic.py

Normative goals for diversification

- Personalized: high accuracy.
- Long-tail: expose users to more low-degree items.
- Bridging: show the other side — but not too polarizing — via weak-tie "bridges".

→ In the code: accuracy metrics · RWED · RWEB (max_distance = "not too far")

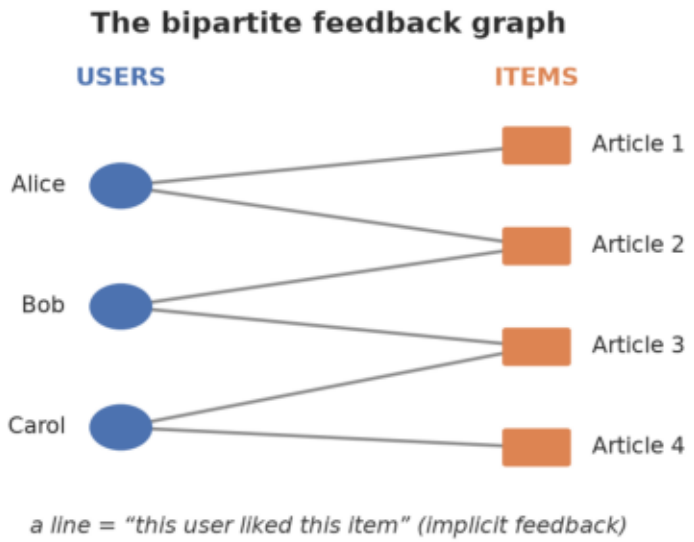
Diversification strategies — the erase matrix Q

- Q on the user-item graph prefers certain random walks over others.
  - Example I — Bridging: items on the opposite side of the user.
  - Example II — Long-tail: low-degree items.
  - Example III — Contrarian: items opposite to the user.
- Generalized framework: any property defined on items or user-item pairs.

→ In the code: RWE(item_erasure=…): RWEB · RWED · or any callable (contrarian = one-liner)

Random Walk with Erasure — the algorithm

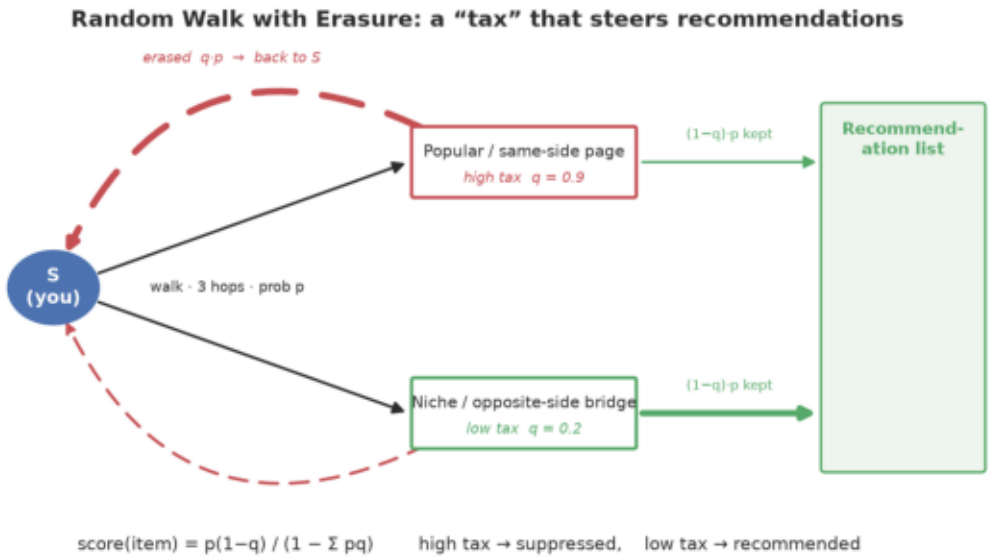
- Bipartite graph: m users, n items; k-step transitions P^k.
- Erase matrix Q ∈ [0,1)^{(m+n)×(m+n)}.
- Sample multiple rounds; erased mass becomes the next round's start.



→ In the code: FeedbackGraph (A^G, P=D⁻¹A^G); item_distribution(users, k)

How erasure steers recommendations

- Score(item) = p·(1−q) / (1 − Σ p·q) — the mass that survives the "tax".
- High tax q ⇒ suppressed; low tax q ⇒ recommended.
- Final score: $w_s = \Sigma_i v_{s,i} P^k \circ (1 - Q)$.



→ In the code: RWE.score_iterative (rounds) == derived closed form; tested to match

Long-tail worked example (reproduced)

- $P^k_{u_4,i^*} = [0.16, 0.16, 0.66]$; $Q = 1 - [1/3, 1/2, 1/2] = [0.66, 0.5, 0.5]$.
- Erased = $P^k \circ Q$; start mass falls $1 \to 0.51 \to 0.26$ across rounds.
- Suppresses the high-degree item, lifts the low-degree ones.
- Our code reproduces these numbers to the decimal.

→ In the code: RWED (β=1) via score_iterative — verified against the slide

Experimental evaluation — three questions

- Q1. Does joint learning identify ideological positions accurately?
- Q2. Can RWE generate accurate AND diverse long-tail recommendations?
- Q3. Can RWE generate accurate AND ideologically diverse recommendations?

→ In the code: IdeologyModel · RWED · RWEB + metrics.py

Datasets & baselines

- 6 Twitter graphs: UK / US / DE × {retweets (RT), URL shares}.
- Plus benchmarks ML-1M and Yelp for long-tail diversity.
- Baselines (RecSys'19 study): Collaborative Filtering, Matrix Factorization,
  - graph-based P^3, and long-tail-diversifying RP^3_β.

→ In the code: ItemKNN · BPRMF · P3 · RP3Beta; data.py (synthetic stand-ins for private data)

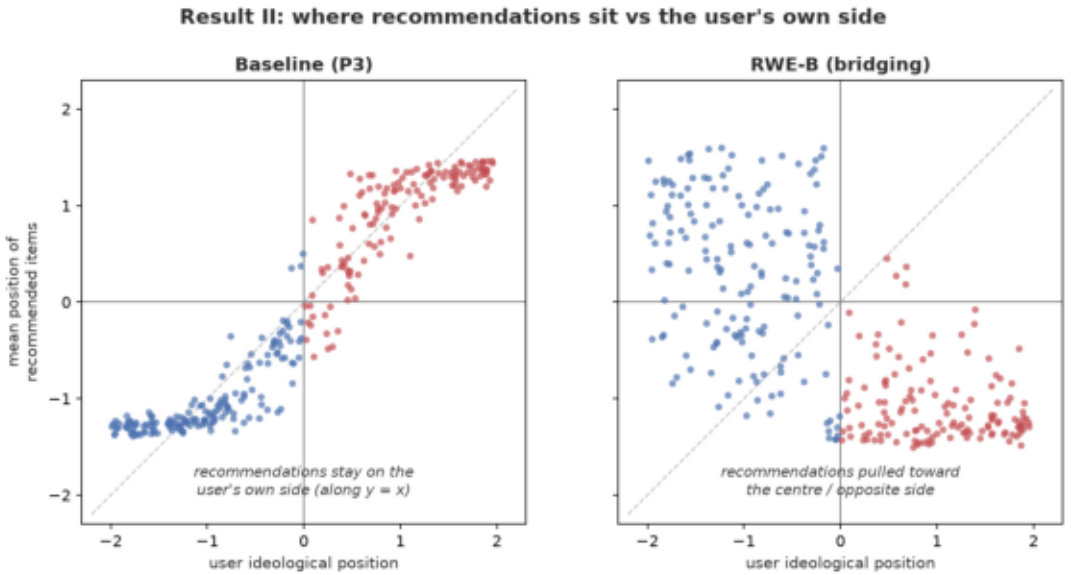
Result I — ideological range of top-10

- Average spread of ideological positions in the top-10 list.
- RWE-B has the widest range on all but the smallest dataset.
  - → more viewpoint-spread per recommendation list.

→ In the code: metrics.rec_range_at_k (reported as rec_range@10)

Result II — ideological diversity

- x = user position, y = mean recommended position (each dot a user).
- Baselines keep users on their OWN side (dots on the diagonal).
- RWE moves recommendations to the OPPOSITE quadrant / centre.



→ In the code: docs/make_diagrams.py quadrant_scatter — metrics.mean_recommended_position

Result III — ideological shift

- User-Shift = Pos(Recs) − Pos(u); Train-Shift = Pos(Recs) − Pos(Train).
- Left users shift right, right users shift left.
- RWE shows the strongest, cleanest shift toward the opposite side.

→ In the code: metrics.ideological_shift (exact definition) → directed_shift

Result IV — weighted ideological diversity

- Two properties: lean toward centre (per user); wider range for extremes.
- Measures UW / TW (weighted by user / training position): Recs, Shift, Range.
- RWE-B is best on all three — while keeping accuracy (AUC, HR@10) on par.

→ In the code: weighted_position (UW/TW-Recs) · weighted_shift · weighted_range — uw_* in evaluate()

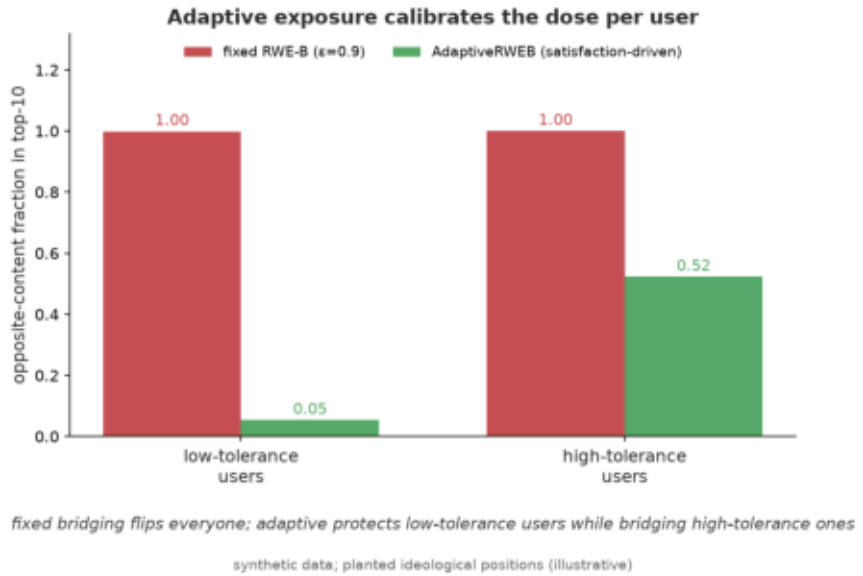
Contributions & main results

- A probabilistic model to estimate ideological positions (users, elites, content).
- Personalized ranking based on RWE.
- Two diversification strategies: long-tail and political bridging.
- A general framework to diversify personalized ranking.

→ In the code: ideology.py · RWE · RWED/RWEB · RWE(item_erasure=…)

Beyond the paper — adaptive exposure

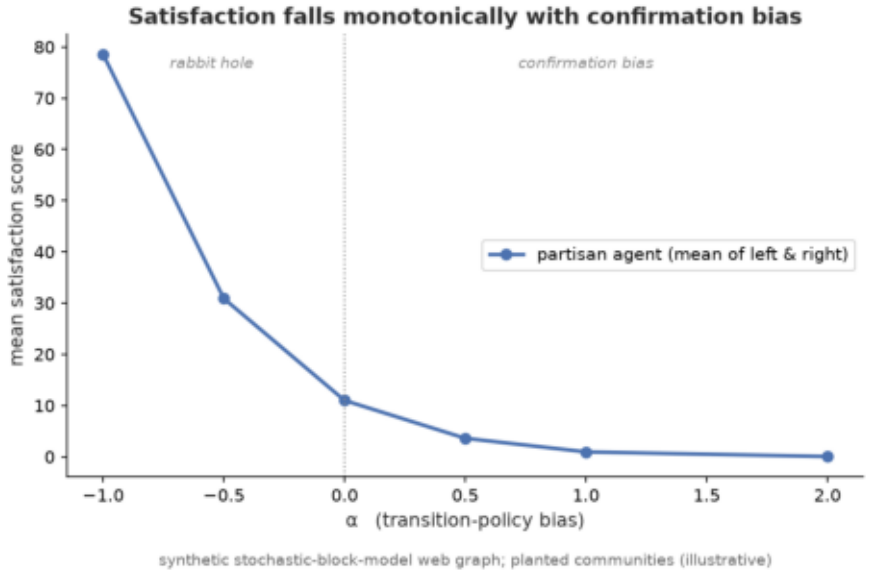
- Our extension: tailor how much opposing content each user sees.
- Satisfaction score = pages read in the first opposing community before exiting.
- Higher tolerance → more opposing content surfaced (AdaptiveRWEB).



→ In the code: rwe/satisfaction.py: WebGraph · SatisfactionModel · AdaptiveRWEB

Beyond the paper — agent browsing simulation

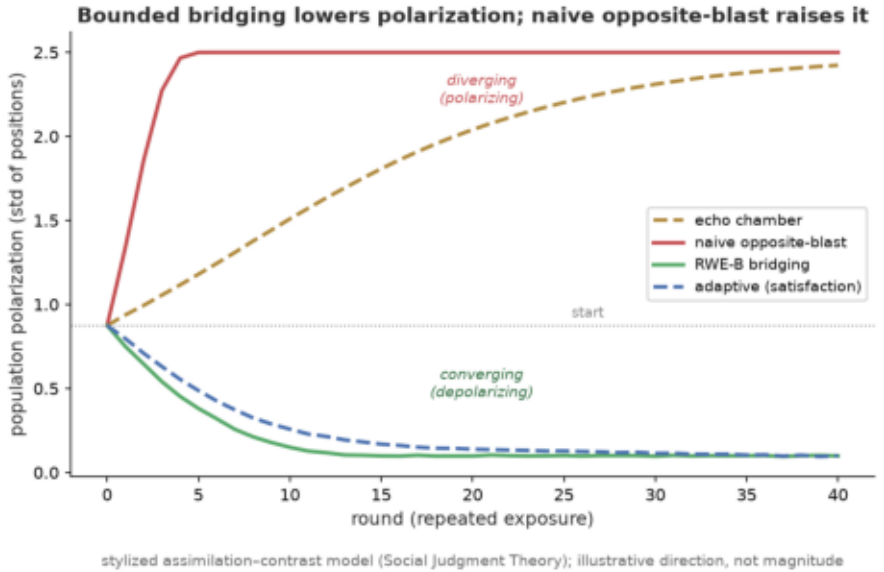
- Agent-based model on a networkx web graph (Louvain/Leiden communities).
- State machine: own side → first opposite page → track → exit.
- Tunable α: confirmation bias (α>0) vs "rabbit hole" (α<0); Monte-Carlo scores.



→ In the code: rwe/agent_sim.py: NewsfeedSimulator · monte_carlo · alpha_sweep

Beyond the paper — does bridging backfire?

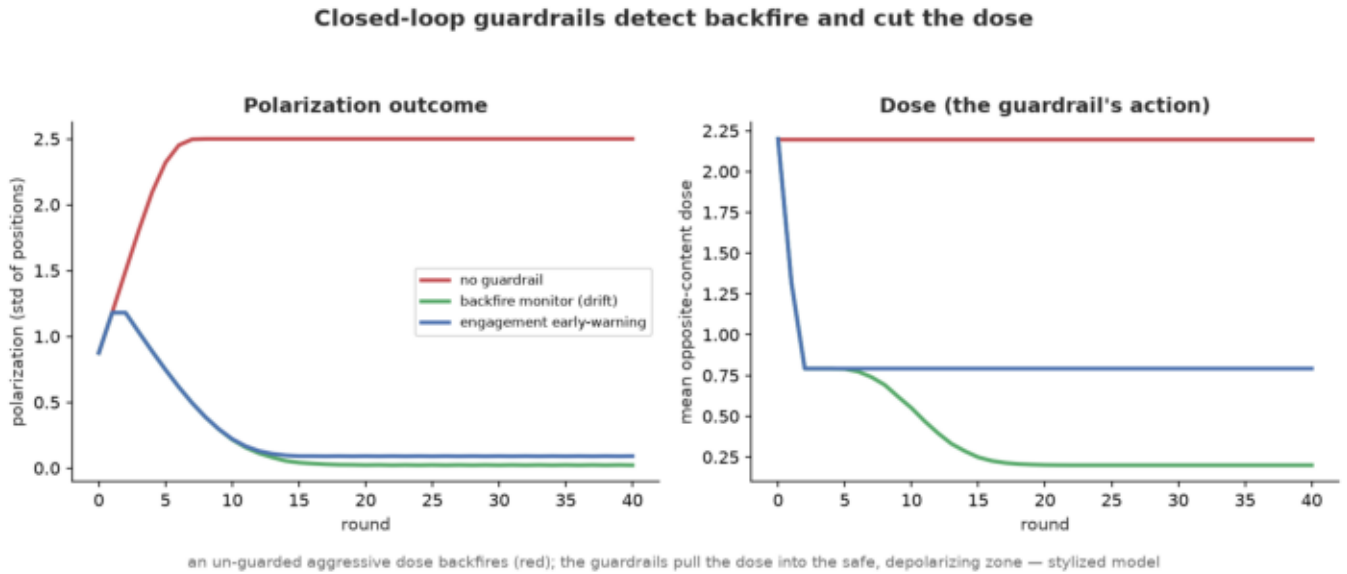
- Risk: naive opposite-view exposure can INCREASE polarization (Bail et al. 2018).
- Simulation: users assimilate nearby views but reject (backfire on) far ones.
- Bounded bridging → opinions converge (depolarize); opposite-blast → diverge.



→ In the code: rwe/opinion_dynamics.py: assimilation–contrast model · compare_policies

Beyond the paper — guardrails that catch backfire

- Don't just bound the dose — MONITOR each user and cut it if backfire starts.
- Backfire monitor (ideology drift) + engagement early-warning (satisfaction).
- Un-guarded aggressive dose polarizes; guardrails detect it and depolarize.



→ In the code: rwe/guardrails.py: BackfireMonitor · EngagementGuardrail · compare_guardrails

This implementation

- Full paper in rwe/ (graph, RWE, ideology, baselines, metrics) + 4 extensions.
- 79 tests — including the worked erasure example reproduced to the decimal.
- 8 diagrams, a beginner guide (GUIDE.md) and a verification report (TALK.md).
- Run: pip install -e . · pytest -q · python examples/demo_synthetic.py

→ In the code: github: greenwichg/random_walks_with_erasure

Thanks

- Paper: Paudel & Bernstein, "Random Walks with Erasure", WWW 2021.
- Original talk: Bibek Paudel — @biasedcoin · bibekp@stanford.edu.
- This deck, diagrams and code-mapping: the implementation project.